

ReactJS HOL- 4

Objective

Create a React application named **blogapp** to demonstrate React component lifecycle methods by fetching blog posts using the Fetch API and displaying them.

Theory

Need and Benefits of Component Life Cycle

React lifecycle methods allow developers to execute code at specific stages of a component such as creation, updating and removal. They are useful for fetching data, managing resources, improving performance and handling cleanup.

Various Lifecycle Hook Methods

Common lifecycle methods include `constructor()`, `render()`, `componentDidMount()`, `componentDidUpdate()`, `componentWillUnmount()`, `componentDidCatch()` and static `getDerivedStateFromError()`.

Sequence of Rendering a Component

1. `constructor()` 2. `render()` 3. `componentDidMount()`. During updates React calls `render()` followed by `componentDidUpdate()`. When an error occurs `componentDidCatch()` is invoked. Before removal `componentWillUnmount()` is executed.

Prerequisites

- Node.js
- npm
- Visual Studio Code

Implementation Steps

1. Create a React project: `npx create-react-app blogapp`
2. Create `Post.js` model class.
3. Create `Posts.js` class component.
4. Initialize state in the constructor.
5. Implement `loadPosts()` using the Fetch API.

6. Call `loadPosts()` inside `componentDidMount()`.
7. Render the list of posts.
8. Implement `componentDidCatch()` for error handling.
9. Import and render `Posts` component in `App.js`.
10. Run the application using `npm start`.

Post.js

```
export default class Post{
  constructor(userId,id,title,body){
    this.userId=userId;
    this.id=id;
    this.title=title;
    this.body=body;
  }
}
```

Posts.js

```
import React,{Component} from 'react';

class Posts extends Component{
  constructor(props){
    super(props);
    this.state={posts:[]};
  }

  loadPosts=()=>{
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res=>res.json())
      .then(data=>this.setState({posts:data}))
      .catch(err=>console.log(err));
  }

  componentDidMount(){
    this.loadPosts();
  }

  componentDidCatch(error){
    alert(error);
  }

  render(){
    return(
      <div>
        <h1>Blog Posts</h1>
        {this.state.posts.map(post=>{
          <div key={post.id}>
```

```

    <h3>{post.title}</h3>
    <p>{post.body}</p>
  </div>
  )}
</div>
);
}
}
export default Posts;

```

App.js

```

import Posts from './Posts';

function App(){
  return <Posts/>;
}

export default App;

```

Output Screenshot:

The application fetches blog posts from <https://jsonplaceholder.typicode.com/posts> and displays each post title as a heading followed by its content. Any runtime error is displayed through an alert dialog.

